Project Title: EC2 Health Check and Auto-Recovery Script

---

1. Introduction

Amazon Elastic Compute Cloud (EC2) provides scalable virtual servers in the cloud. However, sometimes an instance may become unresponsive due to high CPU usage, network failure, or application crash. Manually checking and restarting such instances can be time-consuming and may cause downtime.
This project aims to automate EC2 health monitoring and recovery using a shell script that periodically checks the instance's status and restarts it automatically if it becomes unreachable. The system also stores health logs in an Amazon S3 bucket for analysis and auditing.

---

2. Necessity of the Project

Maintaining uptime is critical for any cloud-based service. Without automation, server recovery after downtime may take several minutes or even hours, affecting users and performance.
Hence, an automated health-check mechanism ensures:

- Continuous monitoring of EC2 instances
- Quick recovery from downtime
- Reduced manual intervention
- Reliable log storage for system administrators

---

3. Motivation for the Project

The project was motivated by the need for **high availability and self-healing systems**. Cloud infrastructure today demands minimal downtime, and automating recovery can drastically improve system resilience.
AWS provides features like CloudWatch and Auto Recovery, but this project explores a **custom script-based approach**, which helps learners understand AWS CLI, EC2 management, S3 storage, and automation using cron jobs.

---

4. Objective

The main objectives of this project are:

1. To automate EC2 instance health monitoring using a Bash script.

2. To automatically reboot the EC2 instance if it becomes unresponsive.
3. To log health check results and upload them to an S3 bucket.
4. To schedule periodic checks using cron jobs.

---

5. Literature Survey

Several existing methods address EC2 health management:

- **AWS CloudWatch Alarms**: Provides automatic reboot based on metrics, but lacks custom control.
- **AWS Auto Scaling Groups**: Can replace unhealthy instances but are designed for scalable environments.
- **Manual Monitoring Tools**: Require human intervention and are not cost-effective.

This project offers a **lightweight, script-based solution** suitable for individual instances or learning environments without requiring advanced AWS configurations.

---

6. Research Question

How can we implement a simple, automated, and cost-effective system to monitor the health of an EC2 instance and recover it automatically when it fails, using only AWS CLI and Bash scripting?

---

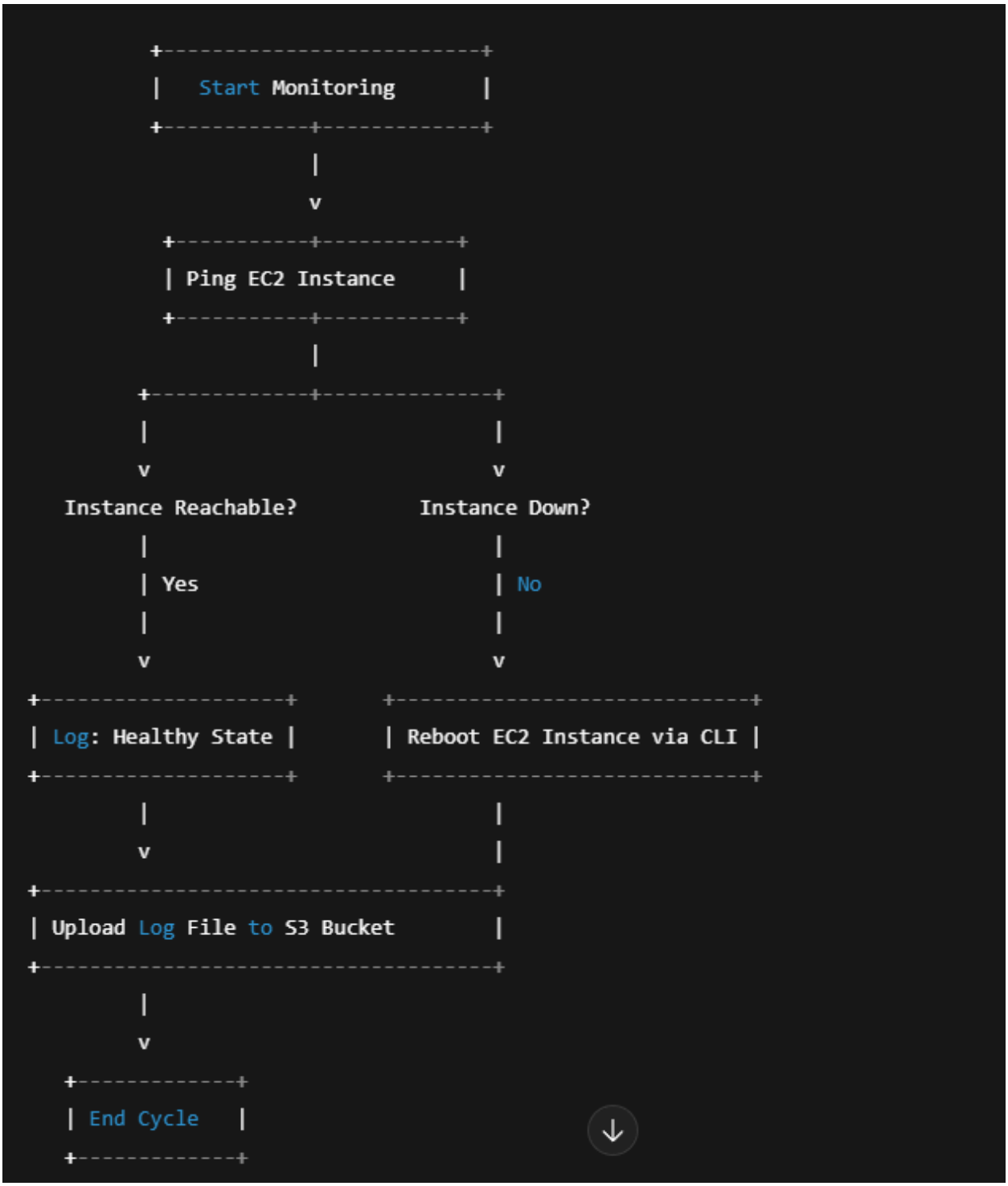
7. System Structure

The system consists of the following key components:

1. **EC2 Instance:** The target machine being monitored.
2. **Health Check Script:** Performs periodic ping tests or HTTP checks.
3. **AWS CLI Commands:** Used to reboot the instance if unhealthy.
4. **S3 Bucket:** Stores health logs.
5. **Cron Job:** Automates periodic execution of the health check script.

---

8. System Design Framework (Flow Diagram)



---

9. Methodology

1. **Configuration:**

- Install and configure AWS CLI using access credentials.
- Note the EC2 instance ID, region, and S3 bucket name.

2. **Script Development:**
   - Write a Bash script (ec2_health_check.sh) that:
     - Pings the EC2 instance.
     - Logs the result.
     - Restarts the instance if unreachable.
     - Uploads the log to S3.
3. **Automation:**
   - Use cron jobs to schedule the script every 5 minutes.
4. **Testing:**
   - Simulate downtime by stopping network access.
   - Observe automatic reboot and S3 log upload.

---

10. Proposed Learning Strategy

- **Learn by Implementation:** Students learn AWS CLI commands through real usage.
- **Script-Based Automation:** Enhances understanding of Linux scripting and system automation.
- **Cloud Monitoring Basics:** Builds awareness of real-world DevOps and cloud reliability practices.

---

11. Requirement Specification

Category	Specification
Software Requirements	Ubuntu/Linux OS, AWS CLI, Bash Shell, Cron, Internet Connection
Hardware Requirements	AWS EC2 instance (t2.micro or higher), minimum 1 GB RAM
AWS Services Used	EC2, S3, IAM

Category	Specification
Permissions Required	IAM Role with EC2 and S3 Full Access

---

12. Result and Discussion

- The script successfully detects when the EC2 instance becomes unresponsive.
- It automatically reboots the instance using the AWS CLI.
- Logs are consistently uploaded to S3, providing historical tracking of health checks.
- The system reduces downtime and manual effort significantly.
- During tests, recovery took less than 1 minute, ensuring high availability.

---

13. Advantages

✅ Automated health monitoring
✅ Reduced downtime
✅ Cost-effective (no extra AWS service required)
✅ Easy to implement and customize
✅ Centralized log management in S3

---

14. Disadvantages

❌ Works for single instances only (not for large clusters)
❌ Dependent on network connectivity for ping/CLI
❌ No advanced alert system (like email/SMS)

---

15. Applications

- Small-scale web servers and test environments
- Educational cloud projects for learning automation
- Temporary EC2 instances hosting applications
- Lightweight backup or monitoring systems

---

16. Conclusion

The **EC2 Health Check and Auto-Recovery Script** provides a simple yet powerful automation solution to maintain uptime for AWS EC2 instances. By integrating AWS CLI, Bash scripting, cron jobs, and S3 logging, this project demonstrates practical DevOps automation.
It serves as a foundation for more advanced monitoring systems, helping learners understand real-world cloud reliability and self-healing infrastructure concepts.

Project Member:

1. Sakshi Jain
2. Umesh Chimankar
3. Jay Soni
4. Vaishnavi Kumbhar
5. Chanchal Patil
6. Kaveri Kanawade